

Spring Boot CRUD Assignment - Question 1

Name: Anshul Patyal
Roll Number: 2024eb01137
Project: Spring Boot JSP JPA CRUD application
GitHub URL: <https://github.com/anshpay/2024eb01137-spring-crud>

1. Approach

I implemented a library management application using two related entities: `Author` and `Book`. The application follows a standard layered Spring Boot structure: entity classes for persistence, repository interfaces for database access, service classes for business logic, controllers for Spring MVC request handling, and JSP pages for the view layer.

2. Entity Relationship Design

Entity	Main Attributes	Relationship
Author	id, name, specialization, email	One author can have many books
Book	id, title, isbn, publishedYear, author	Each book belongs to one author

```
@Entity
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();
}

@Entity
public class Book {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3. Database Population

A `CommandLineRunner` named `DataSeeder` creates 10 authors and 10 books when the application starts. H2 is used as the database and Hibernate creates the tables automatically through JPA annotations.

4. Create Operation

JSP forms are available at `/authors/new` and `/books/new`. The controller accepts form submissions, binds request parameters to entity objects, attaches the selected author to a book, and saves through the service layer. Integrity violations such as duplicate email or duplicate ISBN are caught and shown as form errors.

```
@PostMapping("/books")
public String createBook(@Valid @ModelAttribute Book book,
                        BindingResult bindingResult,
                        @RequestParam Long authorId,
                        Model model,
                        RedirectAttributes redirectAttributes) {
    bookService.save(book, authorId);
    redirectAttributes.addFlashAttribute("success", "Book added successfully.");
    return "redirect:/books";
}
```

Library Manager Books Authors

Add Book

Title

ISBN

Published Year

Author

Aarav Mehta

▼

Save Book

5. Read Operation

The `/books` and `/authors` pages list all rows. The book listing also displays the custom inner join result from the repository layer.

```
@Query("""
select b.id as bookId, b.title as title, b.isbn as isbn,
       b.publishedYear as publishedYear, a.name as authorName,
       a.specialization as specialization
from Book b inner join b.author a order by b.id
""")
List<BookAuthorView> findBookAuthorDetails();
```

Books

Book List

ID	Title	ISBN	Year	Author	Action
1	Spring Boot in Practice	ISBN-2024-001	2021	Aarav Mehta	Update
2	Relational Models	ISBN-2024-002	2020	Isha Sharma	Update
3	Cloud Native Java	ISBN-2024-003	2022	Rohan Kapoor	Update
4	JUnit Essentials	ISBN-2024-004	2019	Neha Verma	Update
5	Secure Web Apps	ISBN-2024-005	2023	Karan Singh	Update
6	Applied ML Systems	ISBN-2024-006	2021	Priya Nair	Update
7	Modern JSP MVC	ISBN-2024-007	2018	Vikram Rao	Update
8	Algorithms Refresher	ISBN-2024-008	2020	Meera Iyer	Update
9	Microservice Patterns	ISBN-2024-009	2022	Kabir Khan	Update
10	DevOps Playbook	ISBN-2024-010	2023	Ananya Bose	Update
11	Assignment Test Book Updated	ISBN-2024-011	2025	Isha Sharma	Update

Inner Join Result

Authors

ID	Name	Specialization	Email	Action
1	Aarav Mehta	Distributed Systems	aarav.mehta@example.com	Update
2	Isha Sharma	Database Design	isha.sharma@example.com	Update
3	Rohan Kapoor	Cloud Computing	rohan.kapoor@example.com	Update
4	Neha Verma	Software Testing	neha.verma@example.com	Update
5	Karan Singh	Cyber Security	karan.singh@example.com	Update
6	Priya Nair	Machine Learning	priya.nair@example.com	Update
7	Vikram Rao	Web Engineering	vikram.rao@example.com	Update
8	Meera Iyer	Data Structures	meera.iyer@example.com	Update
9	Kabir Khan	Microservices	kabir.khan@example.com	Update
10	Ananya Bose	DevOps	ananya.bose@example.com	Update
11	Submission Author Updated	Documentation	submission.author@example.com	Update

6. Update Operation

Update links are provided in the list views. The edit pages load existing records, allow changes, and submit back to controller methods that update the existing database row through the service layer.

```
@Transactional
public Book update(Long id, Book bookDetails, Long authorId) {
    Book book = findById(id);
    book.setTitle(bookDetails.getTitle());
    book.setIsbn(bookDetails.getIsbn());
    book.setPublishedYear(bookDetails.getPublishedYear());
    book.setAuthor(authorService.findById(authorId));
    return bookRepository.save(book);
}
```

7. Testing and Verification

Repository tests verify that the seed data creates 10 rows in each table and that the custom inner join query returns joined book-author details. Service tests use Mockito to verify create and update logic. I also tested the running web app using HTTP requests for list, create, and update operations.

- `mvn test`: 4 tests passed, 0 failures.
- `mvn clean package`: 4 tests passed and executable WAR created.
- Manual HTTP checks: `/books` returned 200, create persisted a new book, update changed it, and the list showed the updated row.

8. Challenges Faced

The main issue was JSP packaging. JSPs are most reliable in a Spring Boot executable WAR with files under `src/main/webapp/WEB-INF/jsp`, so I changed the project packaging from JAR to WAR. Another issue was form validation: the selected author comes from `authorId`, so validation should not require the nested `Book.author` object before the controller attaches it.

9. Conclusion

The project satisfies the requirements for two JPA entities, sample data population, create/read/update operations, custom inner join query, JSP views with CSS, exception handling, service/repository layering, and unit testing.